

EE 587 – INDIVIDUAL PROJECT

HETTIARACHCHI H.A.O.I.A.
E/21/182
SEMESTER 6
15/05/2025

1. The Sobel Filter

1.1 Overview

The Sobel filter is a classic edge detection operator used in image processing to highlight regions of an image where intensity changes sharply. These regions typically correspond to object boundaries and other significant structural features. It was developed by Irwin Sobel and Gary Feldman in 1968 and remains one of the most widely used edge detectors due to its simplicity, low computational cost, and suitability for hardware implementation.

1.2 Principle of Operation

The Sobel filter approximates the gradient of the image intensity function at each pixel. The gradient indicates the direction and rate of the fastest intensity change, and edges correspond exactly to the locations where this change is largest. The approximation is computed using two small 3×3 convolution kernels, one sensitive to horizontal intensity change (G_x) and one sensitive to vertical intensity change (G_y):

Image neighborhood			Kernel G_x			Kernel G_y		
a	b	c	-1	0	+1	-1	-2	-1
d	e	f	-2	0	+2	0	0	0
g	h	i	-1	0	+1	+1	+2	+1

Figure 1. The 3×3 image neighborhood and the Sobel G_x and G_y kernels.

For every interior pixel of the image (border pixels are excluded, since they lack a full 3×3 neighborhood and are forced to zero to match the reference OpenCV/NumPy behaviour), the filter performs four steps

- Extract the 3×3 neighborhood of pixel intensities surrounding the current pixel.
- Convolve this neighborhood separately with G_x and G_y to obtain the horizontal and vertical gradient values.
- Combine G_x and G_y into a single gradient magnitude representing the edge strength at that pixel.
- Saturate the result to fit the output pixel format (8-bit, 0–255).

2. Mathematical Background

2.1 Convolution

Let the grayscale input image be represented as a discrete two-dimensional function $I(x, y)$, giving the intensity at pixel (x, y) . For a pixel at position (x, y) , the Sobel kernels K_x and K_y are convolved with the 3×3 neighborhood centred at that pixel:

$$G_x(x, y) = \sum_{i=-1}^1 \sum_{j=-1}^1 K_x(i, j) \cdot I(x + i, y + j)$$

$$G_y(x, y) = \sum_{i=-1}^1 \sum_{j=-1}^1 K_y(i, j) \cdot I(x + i, y + j)$$

Labelling the nine neighborhood pixels a through i as shown in Figure 1, the convolution reduces to a small set of additions and subtractions

$$G_x = [a + 2d + g] - [c + 2f + i]$$

$$G_y = [g + 2h + i] - [a + 2b + c]$$

G_x is therefore a weighted difference between the left and right columns of the neighborhood it is large when intensity changes sharply from left to right, i.e. across a vertical edge. Symmetrically, G_y is a weighted difference between the top and bottom rows, and responds strongly to horizontal edges.

The Sobel kernel is the outer product of a smoothing vector and a differencing vector

$$K_x = [1, 2, 1]^T \cdot [-1, 0, 1]$$

The $[1, 2, 1]$ component is a discrete approximation of Gaussian smoothing, which suppresses noise, while the $[-1, 0, 1]$ component approximates a first derivative using central differences. This combination makes the Sobel operator noticeably more noise-robust than a naive two point gradient, since it blurs slightly in the direction perpendicular to the derivative before differentiating.

2.3 Gradient Magnitude and the L1 Approximation

The exact gradient magnitude and direction at a pixel are given by

$$G = \sqrt{G_x^2 + G_y^2}, \quad \theta = \arctan\left(\frac{G_y}{G_x}\right)$$

Because square root computation is expensive to implement in hardware, this project uses the L1 norm approximation of the gradient magnitude in place of the true Euclidean (L2) norm:

$$G \approx |G_x| + |G_y|$$

This approximation overestimates the true magnitude in general (except when G_x or G_y is zero), but it preserves relative edge strength well enough for detection purposes, and requires only adders and absolute value logic (two's complement negation with a multiplexer) rather than a square root unit a significant simplification for FPGA implementation.

2.4 Saturation

For 8 bit input pixels, the individual gradient components fall within the range:

$$G_x, G_y \in [-1020, 1020]$$

and their L1 sum can therefore exceed the 8 bit output range. The final pipeline stage clamps the result to fit back into an 8 bit pixel:

$$G_{out} = \min(G, 255)$$

3. Per Pixel Processing Pipeline

Combining the above steps gives the processing pipeline implemented per output pixel, shown in Figure 2. Each interior pixel's 3×3 neighborhood (assembled from the row buffer shift registers in the streaming architecture) is convolved in parallel to produce G_x and G_y , which are then combined via the L1 magnitude approximation and saturated to an 8 bit output value.

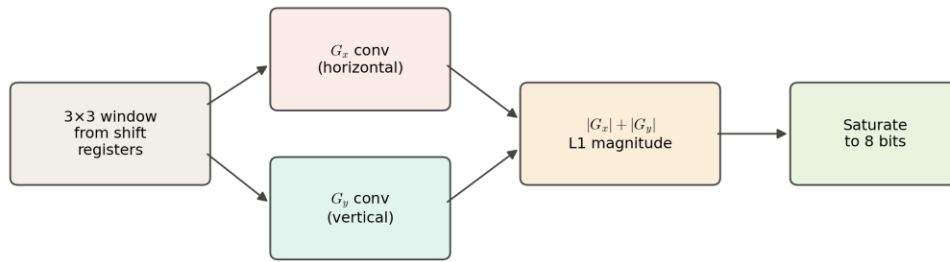


Figure 2. Per pixel Sobel processing pipeline, from neighborhood extraction to saturated 8-bit output.

4. Full Implementation Pipeline

The Sobel edge detector is implemented as a complete hardware software pipeline spanning Python pre/post-processing and a Verilog hardware simulation stage. Figure 3 shows the full data flow, from the input image through to the final pixel-match report, along with the file responsible for each stage and the libraries or tools it depends on.

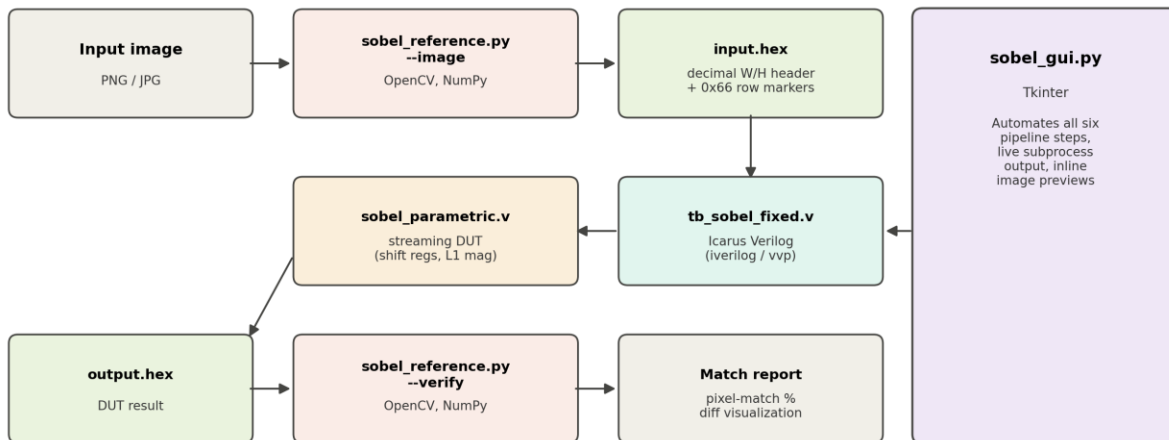


Figure 3. Full pipeline: Python pre processing, Verilog simulation, and Python verification, orchestrated by sobel_gui.py.

4.1 Stage by Stage Breakdown

1. Pre-processing — *sobel_reference.py* (--image mode)

Reads the input image using OpenCV (cv2), converts it to grayscale, and writes it out as input.hex a plain text hex file consisting of a decimal width/height header followed by the pixel data for each row, with a 0x66 byte inserted as a row marker between rows. This file format lets the Verilog testbench read the image using standard \$readmemh style file I/O without needing a custom parser.

- Libraries: OpenCV (cv2), NumPy

2. Hardware simulation — *tb_sobel_fixed.v* driving *sobel_parametric.v*

The testbench parses the header and row markers out of input.hex, streams pixel rows into the parameterized Sobel module (sobel_parametric.v) through shift registers, and captures the resulting edge magnitude pixels. sobel_parametric.v implements the streaming row buffer architecture described in Sections 1–3: 3×3 neighborhood extraction, parallel Gx/Gy convolution, L1 magnitude approximation, and 8 bit saturation. The DUT output is written back out to output.hex in the same header + row marker format as the input.

- Tools: Icarus Verilog (iverilog to compile, vvp to run the simulation)
- Language: Verilog

3. Verification — *sobel_reference.py* (--verify mode)

Reads back output.hex, reconstructs it into an image array, and independently computes the golden reference Sobel result directly in Python using OpenCV's own Sobel/filter functions on the original input image. The two results are compared pixel by pixel to produce a percentage pixel match score and an optional visual difference map, giving an objective, repeatable measure of how closely the hardware implementation tracks the software reference.

- Libraries: OpenCV (cv2), NumPy

4.2 File and Tool Summary

For quick reference, the table below summarizes every file in the pipeline, its role, and the language or library it uses.

File	Role	Language / Library
<i>sobel_reference.py</i>	Pre-processing (--image) and verification (--verify); Python CLI reference pipeline	Python, OpenCV, NumPy
<i>input.hex / output.hex</i>	Intermediate image representation: decimal W/H header + pixel rows + 0x66 row markers	Plain text hex format
<i>sobel_parametric.v</i>	Parameterized streaming Sobel DUT: shift register neighborhood, Gx/Gy convolution, L1 magnitude, saturation	Verilog
<i>tb_sobel_fixed.v</i>	Self checking testbench: reads input.hex, drives the DUT, writes output.hex	Verilog, Icarus Verilog (iverilog / vvp)
<i>sobel_gui.py</i>	Desktop GUI automating all pipeline steps with live output and image previews	Python, Tkinter, PIL

5. Running the Pipeline

This section explains three ways to run the full Sobel pipeline: manually, step by step, using CMD and Vivado; running every step from CMD in one go; and running everything through the `sobel_gui.py` graphical interface.

5.1 Method 1: (Python + Vivado)

This is the manual workflow: generate the input hex file with Python, simulate the design in Vivado's XSIM, then verify the result with Python again.

Step 1 — Create a New Vivado Project

Launch Vivado 2019.2 and select Create Project from the Quick Start menu. Choose RTL Project as the project type.

Step 2 — Add Design and Simulation Sources

Navigate to File → Add Sources → Add or Create Design Sources and add `sobel_parametric.v` as the top level design module.

Navigate to File → Add Sources → Add or Create Simulation Sources and add `tb_sobel_fixed.v`. Placing the testbench under Simulation Sources → `sim_1` ensures it is compiled only during simulation and not treated as synthesisable RTL.

Step 3 — Configure Image Dimensions

The testbench uses ``ifndef` preprocessor guards so that `WIDTH` and `HEIGHT` can be set at compile time. Edit the default values directly in both source files to match the input image dimensions.

In `sobel_parametric.v` (lines 8–9):

```
parameter WIDTH = 1000,  
parameter HEIGHT = 563
```

In `tb_sobel_fixed.v` (near the top):

```
`ifndef WIDTH  
`define WIDTH 1000  
`endif  
`ifndef HEIGHT  
`define HEIGHT 563  
`endif
```

Step 4 — Prepare the Input Hex File

Generate `input.hex` from the source image using the Python pre-processing script:

```
python sobel_reference.py --image photo.jpg --hex_out input.hex --no_blur
```

Vivado's XSIM simulator resolves relative file paths to its working directory. Copy input.hex into

```
<project>.sim/sim_1/behav/xsim/
```

If this folder does not yet exist, run the simulation once to let Vivado create it, copy the file in, then run again.

Step 5 — Run Behavioral Simulation

- Run Simulation → Run Behavioral Simulation

Vivado compiles both source files and launches XSIM. Testbench progress messages appear live in the Tcl Console at the bottom of the window:

```
TB START time=0 WIDTH=1000 HEIGHT=563
Loaded pixels: 563000
Row 1/563, captured out_idx=1000
...
SUCCESS: output.hex written.
```

If the simulation pauses at the waveform viewer before completing, click the Run All button in the simulation toolbar to allow the testbench to run through to its \$finish statement. For a 563,000 pixel image this may take several minutes.

Step 6 — Retrieve output.hex

Once SUCCESS is printed in the Tcl Console, output.hex is located in the XSIM working directory:

```
<project>.sim/sim_1/behav/xsim/output.hex
```

Copy this file back to the main project folder containing sobel_reference.py.

Step 7 — Post-Processing and Verification

Reconstruct and display the edge-detected output image:

```
python sobel_reference.py --hex_in output.hex --show
```

Verify pixel-by-pixel against the Python golden reference:

```
python sobel_reference.py --image photo.jpg --verify output.hex --no_blur
```

5.2 Method 2 — Command Line (CMD) Workflow

An alternative to the Vivado GUI is to run the full pipeline directly from the Windows Command Prompt using Icarus Verilog. This method requires no project setup and is faster for repeated test runs.

#	Action	Command
1	Convert image to hex	<code>python sobel_reference.py --image photo.jpg --hex_out input.hex --no_blur</code>
2	Compile Verilog (iverilog)	<code>iverilog -g2012 -DWIDTH=1000 -DHEIGHT=563 -o sim.out sobel_parametric.v tb_sobel_fixed.v</code>
3	Run simulation	<code>vvp sim.out</code>
4	View DUT output image	<code>python sobel_reference.py --hex_in output.hex --show</code>
5	Verify vs golden reference	<code>python sobel_reference.py --image photo.jpg --verify output.hex --no_blur</code>
6	View output (full Python path)	<code>C:\Python314\python.exe sobel_reference.py --hex_in output.hex --show</code>

Notes

- Replace WIDTH=1000 and HEIGHT=563 with the actual dimensions of the input image as printed by Step 1.
- Step 6 uses the full Python path (C:\Python314\python.exe) to ensure the correct interpreter is used when the system has multiple Python versions installed.
- All commands must be run from the folder containing all project files (sobel_parametric.v, tb_sobel_fixed.v, sobel_reference.py, and input.hex).

5.3 Method 3: Using the GUI

sobel_gui.py wraps all of the above into a single Tkinter application, so no manual CMD commands are needed. To launch it:

```
C:\Python314\python.exe sobel_gui.py
```

Once the GUI opens, the workflow is:

- Select the input image using the file-browser button.
- Click the step to generate input.hex — the GUI calls `sobel_reference.py --image` internally and shows its console output live.
- Click the step to compile and simulate — the GUI calls `xvlog / xelab / xsim` (or `iverilog/vvp`, depending on which backend is configured) and streams the simulator's output into the log pane.
- Click the step to verify — the GUI calls `sobel_reference.py --verify` and displays the pixel-match percentage.

- The GUI shows the input image, the hardware (DUT) output, and the golden reference image side by side as inline previews, so the result can be inspected visually without opening any files manually.

This mode is the fastest way to iterate during development and testing, since it removes the need to type or re type CMD commands for each run and surfaces errors from any stage directly in the interface.

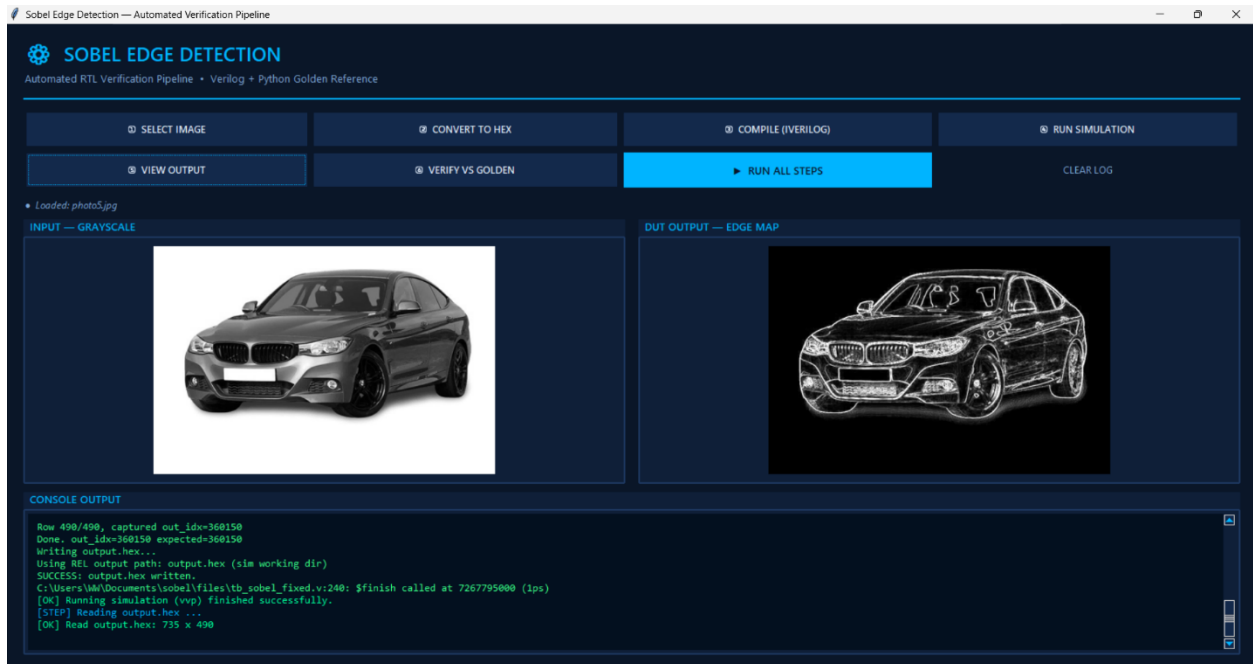


Figure 4. Graphical user Interface for Automate this

REFERENCES

- [1] S. Bleyer, "Icarus Verilog for Windows," *bleyer.org*, 2014. [Online]. Available: <https://bleyer.org/icarus/>
- [2] OpenCV Team, "Sobel Derivatives — OpenCV Documentation," *docs.opencv.org*, 2023. [Online]. Available: https://docs.opencv.org/4.x/d2/d2c/tutorial_sobel_derivatives.html
- [3] V. Kizheppatt, "Image Processing on Zynq (FPGAs): Part 9 Edge Detection through Sobel Operation," *YouTube*, Apr. 04, 2020. [Online]. Available: <https://www.youtube.com/watch?v=TcjqZG2pbHw>
- [4] Pekinese, "Python image (.jpeg) to hex code," *Stack Overflow*, Jul. 22, 2014. [Online]. Available: <https://stackoverflow.com/questions/24889338/python-image-jpeg-to-hex-code>
- [5] "The Sobel Edge Detector using FPGA Project: Sobel Edge Detection Verilog Code," *Kitiyo.com*, 2026. [Online]. Available: <https://edge.kitiyo.com/2009/codes/sobel-edge-detection-verilog-code-full.html>